

REMORA: A Policy-Gated Multi-Oracle Assurance Architecture for Agentic AI

Stian Skogbrott*

Luftfiber AS

<https://github.com/darklordVirtual/REMORA-research>

Preprint v0.10.0, revision 2026-08-07 (synchronized with repository code); repository tag **review-v1** — compiled August 7, 2026 by CI from the repository LaTeX source, synchronized with the code as of the revision date above

Abstract

Autonomous AI agents that invoke tools, query databases, or actuate real-world systems need a gate that decides—before anything executes—whether an action may proceed autonomously, requires verification, warrants abstention, or must be escalated to a human. We present **REMORA** (Reasoning Ensemble Multi-Oracle Routing Architecture), a research-grade control layer that makes this decision. Several independent language models assess each proposed action in parallel; a deterministic policy engine applies hard safety rules that no probabilistic score can override; and every action leaves the gate as one of four auditable outcomes: ACCEPT, VERIFY, ABSTAIN, or ESCALATE.

We report three findings.

First, autonomous safety needs a deterministic floor. On a leakage-controlled tool-call benchmark, REMORA’s full policy gate produced zero simulated unsafe acceptances with significantly higher decision utility and accuracy than heuristic baselines (mean utility +0.456, $p \approx 10^{-4}$; accuracy 90% vs. $\leq 60\%$)—and an ablation shows the zero is attributable entirely to the deterministic hard rules, not to the multi-model voting machinery. Under an intent-gating protocol on 208 independently sourced AgentHarm scenarios [5], the same rule set produced zero false authorizations. Probabilistic consensus improves routing; it cannot override policy.

Second, unknown state is a resolution problem, not a verdict. Absence of a record without a declared completeness guarantee is UNKNOWN, not UNSUPPORTED. We make that distinction operational: a VERIFY must name an actual bounded lookup and be followed by full re-entry of the whole router on a fresh

observation, or it degrades to ABSTAIN—promising a verification that cannot happen is worse than stopping. Under study conditions this recovers read utility from 0% to 100% in a regime where every value verdict is UNKNOWN, with zero corrupt-identifier acceptances and zero cross-tenant lookups.

Third, the routing behaviour transfers to new external data. In one sealed BFCL v4 run (1,527 episodes), REMORA met **all five pre-registered targets**: unknown required inputs were never guessed (0/32), irrelevant tools were refused 258/258 times, unobtainable inputs were refused 98/99 times, obtainable inputs were sent to verification 96/99 times, and well-formed wrong calls were accepted 28/258 times (**10.9%**, target $\leq 20\%$). Labelled routing accuracy was 91.2%. The result confirms the current routing pipeline on unseen data while preserving the distinction between structural validity and semantic task correctness.

Two further findings constrain how much trust to place in model signals: model confidence *anticorrelates* with correctness exactly in the high-disagreement phase where routing matters most, and a pre-registered round on 1,231 fresh items failed to confirm our temperature-based selection signal, which is therefore retained only as a logged diagnostic.

Key limitations: the entropy observable is computed by a token-fingerprint proxy rather than full semantic entropy over NLI clusters; the QA benchmark is partly author-curated; semantic evidence retrieval is pluggable but not live; the audit hash chain is tamper-evident, not tamper-proof, without external append-only storage; and the BFCL v4 track supplies no authoritative task-intent/tool-contract bundle and excludes wrong-argument values because BFCL has no independently vouchable state table. It therefore confirms the complete routing pipeline, not the isolated causal effect of semantic intent matching. REMORA

*Developer and maintainer of REMORA. Luftfiber AS. Contact: support@luftfiber.no.

is a governed autonomy layer, not a replacement for domain authority. The system is **SHADOW_ONLY**: independent external replication and production field evidence remain pending.

Keywords: agentic AI safety, multi-oracle consensus, selective prediction, uncertainty routing, policy-as-code, audit governance, human-in-the-loop.

1 Introduction

The deployment of LLM-powered agents capable of invoking tools introduces a class of decision problem distinct from conversational AI: an agent proposes a *concrete action* in the world, and the system must decide whether to execute it. Unlike a hallucinated sentence, an erroneously executed database write or wrongly triggered process shutdown has immediate, potentially irreversible consequences.

Existing safeguards (RLHF alignment, system prompts, content classifiers) are semantic filters applied at training or prompt time. They are not designed to evaluate a specific proposed action against a specific operational context, evidence base, and regulatory policy at inference time. Majority vote among LLMs improves accuracy but cannot block a confident, wrong consensus from triggering unsafe execution.

We argue that what is needed is not a *better oracle* but an *assurance layer*: a system that evaluates whether the conditions for autonomous action are met before anything is executed. When those conditions are not met, the system must route toward verification, abstention, or human review, recording the reasoning in an auditable envelope.

REMORA demonstrates such a layer. Its core thesis: **governed autonomy requires explicit routing of uncertainty—not suppression of it.**

The contribution of this paper is a **system architecture and empirical evaluation for governed agentic AI decision control**—not a new foundation model, training procedure, or benchmark. It is a reference implementation, a set of measurable observables, a documented set of negative results, and an honest account of what remains unsolved.

A note on terminology. Earlier versions of this paper named its observables after statistical physics (*temperature*, *free energy*, *Lyapunov stability*). That vocabulary has been withdrawn: it invited a physical reading the system does not support, and the quantity it rested on — structural temperature — failed pre-registered confirmation on fresh data (§5, §15). What remains are the information-theoretic quantities the benchmarks report: entropy H and dissensus D over

the weighted verdict distribution, and a trust score τ derived from them. The three consensus regimes (ordered/critical/disordered) keep their names because they name existing code, not a physical state.

2 Background and Related Work

2.1 LLM Ensembles and Self-Consistency

Wang et al. (2023) [7] introduced self-consistency sampling over chain-of-thought paths. Lakshminarayanan et al. (2017) [13] demonstrated that deep ensembles from distinct random initialisations improve both accuracy and calibration. REMORA’s oracle fan-out is structurally related but uses *separately configured oracle endpoints* rather than multiple samples from one model (the 2026-07 clean round and the AgentHarm validation use a Cloudflare Workers AI cross-family trio — Meta LLaMA, Alibaba Qwen, Mistral; earlier rounds used a Groq three-LLaMA swarm), and weights oracles by *inverse pairwise correlation* rather than treating them as exchangeable. Kuncheva & Whitaker (2003) [14] formalised diversity measures for classifier ensembles; REMORA’s inverse-correlation weighting can be understood as an instance of this family applied to instruction-following LLMs. Wang, Aitchison & Rudolph (2023) [55] show that parameter-efficient LoRA ensembles improve predictive accuracy and uncertainty quantification, with ensemble diversity as the key driver; this motivates, but does not independently validate, REMORA’s heterogeneous oracle aggregation.

2.2 Multi-Agent Debate

Du et al. (2024) [8] showed that inter-model debate can reduce hallucination. REMORA’s dissensus metric D is a one-shot, non-iterative analogue: disagreement is measured once, not resolved through iterative exchange.

2.3 LLM-as-Judge and Verifier Models

Zheng et al. (2023) [9] established the LLM-as-judge paradigm. Cobbe et al. (2021) [10] trained outcome verifiers that rank complete model solutions. REMORA differs by using a *structured policy engine* rather than a learned verifier, enabling interpretable hard blocks that cannot be overridden by a confident oracle.

2.4 Selective Prediction and Abstention

Chow (1970) [11] established the optimal error-reject tradeoff for binary classifiers. El-Yaniv & Wiener (2010) [12] extended this to a distribution-free selective classification framework; their coverage-accuracy bound is the foundational result that Geifman & El-Yaniv (2017) [15] apply to deep networks. Kadavath et al. (2022) [16] find promising LLM self-knowledge on tasks resembling training data, with weaker calibration and self-evaluation on novel tasks. REMORA’s abstention is grounded in structural oracle disagreement rather than self-reported confidence.

2.5 Conformal Prediction

Conformal prediction originates with Vovk, Gammerman & Shafer (2005) [54]; Shafer & Vovk (2008) [49] and Angelopoulos & Bates (2021) [17] established it as a practical distribution-free framework for coverage guarantees. REMORA implements Mondrian phase-stratified conformal calibration for the ordered phase, and a CRC-inspired weighted empirical selective router [3] with importance weights (following weighted conformal prediction under covariate shift [4]) for the critical phase (§8.3). The exchangeability violation in the critical phase is documented as a negative result (§15). Conformal methods have since been brought to LLM outputs directly: conformal factuality guarantees via claim back-off [41] and conformal abstention for hallucination mitigation [58]; REMORA’s contribution is narrower and different in object: phase-stratified calibration over multi-oracle governance observables for *action gating*, not per-output factuality.

2.6 Prover-Verifier Games

Kirchner et al. (2024) [2] showed that prover-verifier *training* games improve LLM output legibility by requiring provers to justify claims to iteratively trained verifiers. REMORA does not implement that training game; it borrows the framing for a PVD-inspired offline agreement score, using Semantic Entropy clustering to select the prover (dominant-cluster oracle) and verifier (independent challenger) from already-produced responses, without additional API calls (§8.4).

2.7 AI Governance and Policy-as-Code

Algorithmic-audit scholarship emphasizes institutionalized third-party oversight and audit-system design [45]; Raji et al. identify lack of audit data access as the

most significant vulnerability of the current AI audit ecosystem. REMORA’s DecisionEnvelope addresses this access gap by logging each governance decision with immutable hash-chaining, and the Replay Engine publishes governance episodes for third-party inspection. The EU AI Act (2024) [25] mandates human oversight and audit trails for high-risk AI systems. Open Policy Agent (OPA) [26] provides a production-grade Rego policy engine. REMORA ships an OPA adapter—failing closed to a Python fallback—as a concrete instantiation of policy-as-code for AI decisions; the reference API currently invokes the Python engine directly (§4).

2.8 Assurance Cases

Kelly (1998) [20] and Bloomfield & Bishop (2010) [21] structure safety arguments as claim-evidence-reasoning trees. REMORA’s DecisionEnvelope is designed to populate one node of such a case with a specific gate decision’s evidence chain.

2.9 Human-in-the-Loop / Human-on-the-Loop

Endsley (1995) [19] provides the situation-awareness theory underpinning human supervision of dynamic systems; the automation-oversight literature distinguishes human-in-the-loop confirmation from human-on-the-loop monitoring. REMORA’s four-outcome schema implements this spectrum: ACCEPT (human-on-the-loop) through ESCALATE (human-in-the-loop), with VERIFY and ABSTAIN as intermediate states.

2.10 Industrial AI Safety

The oil-and-gas sector has established quantitative risk frameworks (NORSOK D-010:2021 [43]; IEC 61511:2016 [39]) that define action classes requiring human sign-off. REMORA’s case study (§13) uses this domain as an illustration of how policy-gated AI governance maps to existing industrial safety requirements. No domain-specific correctness is claimed.

2.11 Runtime Guardrails: Agent Permissioning, and AI Control

Content guardrails. Llama Guard and its successors [40] classify conversational inputs/outputs against a safety taxonomy; NeMo Guardrails [47] provides programmable dialogue rails; Constitutional Classifiers [51] defend against universal jailbreaks with classifiers trained on constitution-generated data; Rule-Based Rewards [42] move safety rules into training. These

systems govern *content*; REMORA governs *actions*: its unit of decision is a proposed tool call with operational consequences, and its deterministic hard-block layer is, by design, not overridable by any learned component. A trained classifier and a policy floor fail differently, and REMORA’s position is that action gating in consequential environments needs the latter beneath the former.

Agent-level guardrail systems. LlamaFirewall [28] is the closest industrial system in scope: a layered agent guardrail (prompt-injection detection, alignment checking, code screening). GuardAgent [57] generates guardrail code with a guard LLM; AgentSpec [56] defines a DSL of runtime triggers/predicates/enforcement near-isomorphic to REMORA’s policy invariants. Two 2026 systems share REMORA’s learning ambition: AgentTrust [59] splits threats into lexical (deterministic rules) and semantic (LLM judge with a guarded verdict cache), and *self-distills judge decisions into its rule layer*; Membrane [29] evolves a contrastive safety memory against jailbreaks. REMORA’s differences are deliberate: (1) the hard-block floor is fixed, versioned, and hash-provenanced policy-as-code, learned outputs never become enforcement rules, and the learning layer (AROMER) is monotonicity-tested to *never weaken* policy (`tests/policy/test_governance_intelligence_never_weakens_policy.py`), whereas AgentTrust’s distillation pipeline makes learned judgments load-bearing for enforcement; (2) uncertainty is structural (multi-oracle disagreement with conformal abstention), not a single judge’s confidence; (3) every decision emits a signed, hash-chained DecisionEnvelope. Conversely, AgentTrust reports zero benign hard-blocks across 45,000 actions, the opposite end of the safety/friction trade-off from REMORA’s FBR=100% AgentHarm corner point (§11), and an explicit target for REMORA’s calibration roadmap.

Permissioning and security by design. Progent [52] enforces programmable least-privilege on agent tool calls, reducing attack success from 39.9% to 1.0% on AgentDojo *while maintaining utility*; CaMeL [33] defeats prompt injection by construction via control/data-flow separation, retaining 77% task completion with provable security. These results define the current (safety, friction) frontier and preempt any reading of REMORA’s zero-FAR corner point as sufficient: REMORA’s contribution is not the corner but the governance machinery around it: uncertainty-routed escalation, auditability, and claim-governed evaluation. Capability-based enforcement in the Progent/CaMeL style is complementary and

is the natural enforcement substrate for REMORA’s VERIFY/ESCALATE outcomes.

Benchmarks and environments. Beyond AgentHarm [5], the field evaluates on AgentDojo [32], ToolEmu [48], InjecAgent [62], R-Judge [61], and τ -bench [60]. ToolEmu’s LM-emulated sandbox anticipates the counterfactual-execution idea behind REMORA’s Shadow Mode; REMORA’s replay engine is therefore *not* conceptually unique: its distinction is the governance framing (committed replay artifacts under claim governance). Replication of REMORA’s results on these independently constructed environments is required future work and the honest boundary of every internal benchmark claim in this paper.

AI control. Greenblatt et al. (2024) [36] formalize safety protocols that remain effective when the *policy model itself* is intentionally subversive, using trusted monitoring with limited audit budgets. In their taxonomy REMORA is a trusted-monitoring control protocol: a deterministic trusted layer plus weaker trusted monitors (the oracle swarm) gating an untrusted agent, with ESCALATE as the audit budget. REMORA’s threat model (injection, oracle compromise) does not yet include a strategically subversive agent: collusion and monitor-evasion failure modes from the control literature are open items for the red-team plan.

Regulatory operationalization. COMPL-AI [37] provides a technical interpretation of the EU AI Act as model-level benchmarks; REMORA operationalizes a complementary slice at runtime, Article 12 (logging) via the DecisionEnvelope hash chain, Article 14 (human oversight) via VERIFY/ESCALATE routing, without claiming conformity assessment.

Multi-layer guardrail taxonomies. A systematic literature review of runtime guardrails [50] frames multi-layer defense as a Swiss-cheese model; a layered governance architecture with OS-level enforcement is developed in Ge (2026) [35]; confidence elicitation by fidelity is treated in Zhang et al. (2024) [63]; and formal verification of behavioral safety properties for neural policies in Corsi et al. (2021) [30]. REMORA’s position relative to these four is that it composes recognized layers (policy floor, ensemble uncertainty, conformal abstention, audit chain) with a claim-governance methodology, rather than introducing a new layer type.

3 Problem Statement

Let $\mathcal{A}$ be a space of agent-proposed actions, q a query, and $c = (\text{domain}, \text{risk}, \text{env})$ an operational context. We seek a control function:

$$\Gamma : (q, a, c) \longrightarrow g, \quad (1)$$

$$g \in \{\text{ACCEPT}, \text{VERIFY}, \text{ABSTAIN}, \text{ESCALATE}\}$$

with five required properties: (1) safety-first (hard policy violations map to ESCALATE regardless of consensus); (2) calibration-aware [18] (abstain when uncertainty exceeds threshold); (3) evidence-sensitive (evidence can resolve or contradict ambiguous consensus); (4) auditable (every call produces a hash-chained evidence record, HMAC-signed when a signing key is configured); (5) conservative by default (fail closed toward VERIFY or ESCALATE depending on risk tier).

This is not a question of oracle *accuracy*—REMORA does not know whether action a is correct. It is a question of *assurance conditions*: whether the conditions that justify autonomous execution of a are verifiably met.

4 REMORA Architecture

REMORA processes each gate request through six decision stages, followed by DecisionEnvelope emission (Stage 7). The architecture diagram in the supplementary material (Mermaid) shows the full pipeline. Stage 7 is the output record, not a decision stage.

Stage 1: Intake & Risk Classification. The system classifies intent domain, risk tier ($\in \{\text{low}, \text{medium}, \text{high}, \text{critical}\}$), action type, and target environment. An adversarial admission firewall checks for injection patterns (“ignore previous instructions”, “drop table”, etc.) and fires an immediate ESCALATE if detected.

Stage 2: Oracle Fan-Out. n configured oracle models are queried in parallel via `ThreadPoolExecutor` (the engine accepts $n \geq 2$; the design recommendation is $n \geq 3$ from distinct model families. The 2026-07 clean round and AgentHarm use a Cloudflare Workers AI cross-family trio — Meta LLaMA, Qwen, Mistral — enforced by `remora/oracles/families.py`; earlier rounds used a Groq three-LLaMA swarm whose within-family correlation gave an effective diversity closer to two). Responses with non-null `error` fields are excluded before any aggregation. $\mathcal{O}' \subseteq \mathcal{O}$ denotes the valid oracle set.

Stage 3: Canonicalization & Weighted Consensus. Each response is canonicalized by $\varphi : \text{raw} \rightarrow v$ where v is the tuple (`polarity`, `claim_hash`, `magnitude`, `tags`). The `claim_hash` field uses a pluggable backend: the default `TokenFingerprintBackend` applies a 16-char SHA-256 truncation over NFKC-normalized sorted tokens for backward-compatible, dependency-free operation; the `NLISemanticBackend` replaces this with bidirectional cross-encoder entailment clustering, implementing Semantic Entropy [1]. Oracle weights are derived from inverse pairwise correlation (see §5).

Stage 4: Consensus Observables. Entropy H is computed over the diversity-weighted verdict distribution $\hat{p}$. The `NLISemanticBackend` grounds H as Semantic Entropy [1], $\text{SE}(x) = -\sum_{c \in \mathcal{C}} p(c|x) \log p(c|x)$ over NLI-derived semantic equivalence clusters $\mathcal{C}$. *Note on experimental backend:* all benchmarks reported in §10 use the default `TokenFingerprintBackend` (NFKC-normalised, sorted-token SHA-256 truncation), which approximates cluster identity via lexical overlap. The NLI backend is a drop-in replacement for future work but was not used in the reported results; claims about Semantic Entropy properties should be read accordingly. Dissensus D , consensus concentration η and trust score τ are computed (see §5), and a regime $\in \{\text{ordered}, \text{critical}, \text{disordered}\}$ is assigned. These run on the research and benchmark path; the governed server routes leave them unset (§5).

Stage 5: Evidence Router (Critical Phase). If phase = critical, the `CriticalEvidenceRouter` evaluates a five-field `EvidenceSignal` and returns one of: `evidence_accept`, `abstain`, or `escalate`. *Important:* the current `EvidenceSignal` is an oracle-proxy (built from consensus statistics); semantic BM25/NLI retrieval is pluggable but not live (§8).

Stage 6: Policy Engine. Seven *headline* policy controls are evaluated in priority order before any routing logic; the shipped engine’s complete priority-ordered gate inventory is larger (schema, forbidden-tool, coercion, tainted-argument, rollback, production-write, quorum, trap-detection, session/fleet-risk gates, among others) and is maintained in `remora/policy/decision_engine.py`, with `hard_guard_floor()` as the single deterministic floor no probabilistic result can override. An OPA/Rego adapter with Python fallback is implemented and parity-tested, but the reference API invokes the Python policy engine directly; OPA remains an integration option and production

dependency (fail closed either way). Gate outcome g is assigned.

Output: DecisionEnvelope v2. An attribute-frozen envelope (treated as read-only; nested collections are not deep-frozen) records: request, assessment, gate, reviewer context, follow-up request, case history, policy-learning candidate, and audit hash-chain entry.

4.1 The Argument-Resolution Ladder

The six stages above describe how an action reaches a verdict. They do not describe what the engine reasons about when the action is well-formed but the *state it refers to* is uncertain — which, in operational deployments, is the common case. That reasoning is a distinct ladder, evaluated after every blocking gate so it can only ever make a would-be ACCEPT more cautious, never unblock one:

1. **Authoritative tool metadata.** Effect, risk class, resource type and validator bindings come from a server-side registry. A caller may raise risk, never lower it. Verb heuristics are a fallback for unregistered tools only — a lesson learned when `assign`, `close` and `approve` were absent from a mutating-verb list and thirty writes scored as reads.
2. **Coverage declaration.** A state index may not infer its own completeness. Closed-world authority is a signed, curator-bound declaration scoped per argument role, with a snapshot hash, an as-of date and a freshness bound. Without it, absence means UNKNOWN.
3. **Tenant, hash and freshness binding.** A declaration that does not match the snapshot it claims, or that has aged past its bound, is refused rather than trusted. Cross-tenant consultation is refused outright.
4. **Argument support.** Does the system of record contain this value? Tri-state: present, confirmed absent, or outside coverage. Unknown must never behave like a negative.
5. **Value grounding.** Are the argument values traceable to *this* task — named in the task text, declared by the tool’s own schema, or vouched for by the system of record? A call whose values anchor to nothing here is observationally a foreign call.
6. **Validation requirement.** An argument that steers where the action lands must be confirmed before autonomy, decided by what the argument *does* rather than by whether an index happens to cover it.
7. **Resolver availability.** If a bounded lookup can close the gap, the verdict is VERIFY carrying a `ResolutionPlan` that names the resolver, the exact arguments it may write, and the sources permitted to supply them. If no resolver exists, the verdict is ABSTAIN: promising a verification that cannot happen is worse than stopping.
8. **Validator execution and full re-entry.** The resolver returns one value and hands control back. It never performs the original call and cannot redirect it. The *whole* router then re-runs on a fresh observation, so every guard applies again to the resolved call rather than being patched around.
9. **Read/write posture.** A verified read may proceed autonomously; a write does not, whatever the state evidence says.

This ladder, not the consensus machinery, is what the operational results in §11.5–§11.6 measure. Its tri-state discipline is the recurring commitment: UNKNOWN is a first-class verdict distinct from UNSUPPORTED, and a signal that cannot be established is left absent rather than guessed — a fabricated boolean enters the policy contract as fact, while `None` correctly says nothing establishes this.

5 Consensus Observables

Earlier versions of this paper presented these observables under thermodynamic terminology (structural temperature, free energy, a Lyapunov observable). That framing has been withdrawn: it invited a physical reading the system does not support, and its central quantity did not survive pre-registered testing. What remains are the information-theoretic quantities the benchmarks actually report; the withdrawn material is retained in the negative results (§15), not deleted.

5.1 Weighted Support Distribution

Let $\hat{p}(v) = \sum_{o \in \mathcal{O}} w(o) \cdot \mathbf{1}[\varphi(o) = v]$ be the diversity-weighted support for verdict v , normalized to sum to 1. The diversity weight $w(o)$ penalizes oracles that frequently agree with others:

$$w(o) = \frac{(1/n)}{1 + \sum_{j \neq o} \rho(o, j)} \cdot Z^{-1} \quad (2)$$

where $\rho(o, j)$ is the rolling pairwise agreement rate (window 200 samples) and Z normalizes. Oracles that

agree with the swarm receive lower weight; independent dissenters receive higher weight. The rolling window persists for a long-lived engine instance (as in the offline experiments); the reference API constructs a fresh engine per request, so cross-request persistence of correlation state is a deployment integration point, not a current runtime property.

5.2 Core Observables

Shannon entropy (computed over the token-fingerprint proxy clustering in all reported experiments, not NLI-based semantic clusters; backend note in §4, limitations in §16):

$$H = - \sum_v \hat{p}(v) \log_2 \hat{p}(v) \quad (\text{bits}) \quad (3)$$

Dissensus:

$$D = 1 - \max_v \hat{p}(v) \quad (4)$$

Order parameter:

$$\eta = \frac{\max_v \hat{p}(v) - 1/k}{1 - 1/k} \quad (5)$$

where $k = |\{v : \hat{p}(v) > 0\}|$ is the number of active verdicts.

Trust score:

$$\tau = \eta \cdot (1 - h_{\text{bound}}) \cdot w_{\text{phase}} \cdot \left(1 + \frac{\chi}{\chi_0}\right)^{-1} \quad (6)$$

where h_{bound} is a *heuristic false-consensus risk proxy* from inter-oracle agreement — the implementation uses $q^{n/2}$ (not the proven exponent) and clamps ρ at 0.49, so it can understate correlated-failure risk and must not be cited as a mathematical bound — $w_{\text{phase}} \in \{1.0, 0.5, 0.1\}$ is a regime weight over the three consensus regimes (ordered/critical/disordered), and χ is a scale factor that is algebraically constant in the live path unless internal clamps bind. As a standalone difficulty predictor χ failed (AUC = 0.39, below chance; §15). Both are retained because committed benchmark artifacts depend on them; neither is presented as a validated mechanism.

5.3 What these observables do not do

Two boundaries matter more than the definitions above, because the withdrawn framing obscured both.

They are not populated on any governed path. `servers/api.py` never references the consensus state; `servers/execution_api.py` passes `trust_score=None` and `phase=None` explicitly, and

`build_full_observation` — the builder shared by the assess and execution routes — leaves these fields at their `None` defaults. The engine therefore fails toward `VERIFY/ABSTAIN` there: no probabilistic `ACCEPT` can fire from these signals in the governed path at all. The runtime safety floor is the deterministic hard-block layer (§6). Everything in §11 is measured on the research and benchmark path.

The regime assignment rests on a falsified signal. Regimes were derived by comparing a structural temperature estimate against a calibrated critical value. That signal failed its pre-registered confirmation on fresh data: on $N = 1,231$ held-out items it ranked *worse* than calibrated confidence (AURC 0.0954 vs. 0.0664, paired CI excluding zero; CLAIM-012). Temperature is diagnostics-grade, not an authoritative selector, and the derivation now lives in the negative results (§15) rather than here as method.

6 Decision and Policy Model

6.1 Four Gate Outcomes

Table 1: Gate outcomes and autonomous action status.

Outcome	Meaning	Autonomous action
ACCEPT	Assurance conditions met	Permitted
VERIFY	Plausible, validation required	Pending
ABSTAIN	Uncertainty too high to decide	Blocked
ESCALATE	Human review required	Blocked

ACCEPT does not mean the action is *correct*—it means the assurance conditions for autonomous execution are met. ABSTAIN does not mean the action is *wrong*—it means the conditions for deciding are not met. Both ABSTAIN and ESCALATE block execution; ESCALATE additionally triggers a human-review workflow.

6.2 Hard Blocks (Priority Order)

Policy must override model consensus. Majority vote is never sufficient for critical autonomous action when a hard block condition is met.

6.3 OPA/Rego Integration

REMORA exports a 67-field `PolicyObservation` dataclass as JSON to the OPA endpoint. On any communication error (connection refused, timeout, parse failure), the Python fallback engine applies identical rules and records `source_of_decision =`

Table 2: Priority-ordered policy gates: unconditional hard blocks (1–3, `hard_guard_floor()`) and conditional policy gates (4–7, `_CONDITIONAL_GATES`). Both sets take precedence over the probabilistic ACCEPT decision; only the unconditional blocks are evaluated before all routing logic.

Pri.	Condition	Reason Code	Action
<i>Unconditional hard blocks (<code>hard_guard_floor()</code>)</i>			
1	Adversarial detected	ADM_FIREWALL	ESC.
2	Counterfactual failed	CF_FAILED	ESC.
3a	Evidence contradicted + contradiction cycles	EV_CONTRAD.	ESC.
3b	Evidence contradictions	EV_CONTRAD.	ABS.
<i>Conditional policy gates (<code>_CONDITIONAL_GATES</code>)</i>			
4	Refuse parametric + no ev.	THM_REQ_EV.	VER.
5	Distribution shift	DIST_SHIFT	VER.
6	Phase=critical + risk=crit.	CRITICAL_PHASE	ESC.
7	High/crit. risk + no ev.	EV_INSUFF.	VER.

"python_fallback" in the envelope. No decision is ever withheld due to a missing policy engine.

7 Governance Intelligence Layer: Misspecification-Aware Pre-Policy Enrichment

The policy engine is only as honest as the metadata it is given: an integration bug, or a compromised agent, can label a destructive action as a low-risk read and present the gate with nothing to fire on. Added at package version 0.9.0 (repository tag `review-v1`), the *Governance Intelligence Layer* (`remora/governance_intelligence/`) closes this gap with an opt-in, fully deterministic enrichment stage that runs before any policy rule (no LLM calls): (1) fail-closed normalization of caller-supplied labels (unknown is explicit, never coerced to a safe value); (2) action-semantics extraction from the proposed action text and tool metadata; (3) misspecification-risk inference from label/semantics disagreement; (4) causal-consequence signals [27, 44] (blast radius, expected loss); and (5) policy-generalization risk—would repeatedly accepting this *class* of action remain safe fleet-wide?

Signals merge into the existing `PolicyObservation` under a **strengthen-only** rule: inferred higher risk may override a supplied lower label, never the reverse, and hard-block flags are never cleared. A grid property test across 2,160 observation combinations verifies that enrichment never converts a rejection into an acceptance (`tests/policy/test_governance_intelligence_never_weakens_policy.py`). The

engine remains the sole decision authority.

Table 3: Governance-intelligence benchmark: 50 deterministic tasks across 10 categories of mislabelled, underspecified, or repeated actions, evaluated offline with a deliberately permissive trust baseline so that enrichment is the only barrier to acceptance.

Metric	Result
Unsafe accept rate (no-accept tasks)	0.0%
Metadata-mismatch detection	75%
Unknown-metadata review rate	100%
Legitimate reads still accepted	100%
Escalation precision	0%
Policy-generalization detection	62.5%

The safety-relevant rows hold (0% unsafe accepts; all unknown-metadata and legitimate-read cases routed correctly), but the enrichment layer’s *precision* is weak on this 50-task eval: escalation precision 0% (its escalations did not match the expected set), 75% metadata-mismatch detection, 62.5% policy-generalization detection. These are the committed, deterministic artifact numbers (re-verified); the layer is an offline, opt-in helper, not a validated detector.

Artifact: `artifacts/governance_intelligence/evaluation_results.json`; runner: `experiments/evaluate_governance_intelligence.py`. Without enrichment, `DROP TABLE users` labelled `action_type=read`, `risk_tier=low` reaches ACCEPT on a high-trust path; with enrichment it escalates. The layer is heuristic and English-centric; pattern tables can be evaded by novel phrasing, so it reduces reliance on caller labels rather than replacing verification, evidence, or human review.

8 Evidence-Grounded Critical-Phase Routing

8.1 The Critical-Phase Problem

In the critical phase, oracle consensus is near the phase boundary. Empirically, higher trust scores *negatively* correlate with correctness ($n = 32$ real-oracle items): Q4 high-trust achieves 50% correct; Q1 low-trust achieves 75% correct. Conformal calibration at 5% risk target fails completely on critical-phase items (mean observed risk = 100%, coverage $\rightarrow$ 0%). Standard trust-based routing cannot safely gate critical-phase decisions.

8.2 PhaseAwareGuardrail: Exploiting Trust Inversion

The inversion is a *selection criterion reversal*, not an accuracy problem. Items with $\tau < 0.10$ in the critical phase achieve 76.2% accuracy ($N = 21$) vs. 36.4% for high- τ critical items ($N = 11$). A **PhaseAwareGuardrail** exploits this by applying an inverted score $\tilde{\tau} = 1 - \tau$ for critical-phase items, then calibrating a conformal threshold on $\tilde{\tau}$. A hard gate rejects all items with $\tau \geq \tau_{\max} = 0.10$ (the groupthink boundary) regardless of the conformal result.

The combined pool—all 99 ordered-phase items (sorted by $\tau \downarrow$) plus 21 low- τ critical items ($\tau < 0.10$, sorted by $\tilde{\tau} \downarrow$)—achieves:

- 20.0% coverage ($N = 108$): 85.2% accuracy CI [77.3%, 90.7%]
- 22.1% coverage ($N = 120$): 85.0% accuracy CI [77.5%, 90.3%]

This is a +3.9 pp coverage gain over the ordered-only operating point (18.2%, 86.9% accuracy), with a 1.9 pp accuracy cost—still +43.8 pp above the 41.18% full-coverage baseline. The guardrail is implemented in `remora.selective.guardrail.PhaseAwareGuardrail` and fully tested (25 unit tests).

8.3 Conformal Risk Control Under Covariate Shift

A weighted split-conformal router inspired by Conformal Risk Control is implemented and tested, but it is an offline research component: the reference API does not invoke it, and the implementation does not instantiate the CRC theorem’s hypotheses, so its output is a heuristic bound rather than a certified one. Because it is neither part of the operative architecture nor a valid instance of the formal method, the derivation, the theorem statement, and the reasons no reweighting repairs the gap are given in Appendix E rather than here.

8.4 PVD-Inspired Offline Agreement Score for Critical-Phase Routing

Kirchner et al. (2024) trained prover-verifier games that produce more legible and reliable LLM outputs: helpful and sneaky *provers* justify claims to iteratively retrained small *verifiers* [2]. REMORA does **not** implement that training game. The component below is a PVD-*inspired*, post-hoc agreement score computed over already-produced oracle responses: it selects two

existing responses, scores their agreement with the configured entailment backend (default: token-fingerprint equality, not NLI), and applies a fixed round decay. No model training, live interaction, argument generation, or verifier update occurs, and it is not invoked by the reference API (offline research component).

Protocol.

1. **Cluster** oracle responses via Semantic Entropy clustering [1, 34] (§5), producing semantic equivalence classes $\mathcal{C} = \{C_1, \dots, C_K\}$ sorted by mass.
2. **Prover** = the oracle whose response belongs to the dominant cluster C_1 and has the highest per-oracle confidence.
3. **Verifier** = the oracle with the highest confidence *outside* C_1 (the independent challenger).
4. **Deliberation** ($r = 1, \dots, n_{\text{rounds}}$): evaluate bidirectional NLI entailment $e_r = \text{NLI}(\text{prover} \rightarrow \text{verifier}) \cdot \gamma^{r-1}$ with round-decay factor $\gamma = 0.85$.
5. **Legibility score**: $\mathcal{L} = \bar{e} \cdot |C_1|/N$, where $\bar{e}$ is the mean entailment and $|C_1|/N$ is the dominant cluster mass.
6. **Final confidence**: $\hat{c} = \sqrt{|C_1|/N \cdot c_{\text{verifier}}}$ (geometric mean of prover support and updated verifier confidence).

Routing signal. The PVD confidence is blended with the raw REMORA trust score:

$$\tilde{\tau} = (1 - w_{\text{pvd}}) \cdot \tau + w_{\text{pvd}} \cdot \hat{c}, \quad w_{\text{pvd}} = 0.40 \quad (7)$$

In offline experiments this score can promote borderline critical-phase items to ACCEPT when agreement confidence is high, extending coverage beyond the **PhaseAwareGuardrail** operating point. Neither component is invoked by the reference API, and no formal risk bound applies to this heuristic blend (§8.3).

Implementation. The deliberation entry point is

```
remora.selective.pvd.deliberate(  
    oracle_responses, oracle_confidences,  
    n_rounds=2)
```

It returns a `PVDResult` with fields `prover_cluster_mass`, `legibility_score`, `agreement`, and `final_confidence`. `pvd_routing_score(trust_score, pvd_result)` implements Eq. 7. Fully tested (33 unit tests, no external model calls).

8.5 CriticalEvidenceRouter

The router evaluates a five-field `EvidenceSignal` with fields (`evidence_strength`, `contradiction_score`, `citation_coverage`, `cross_evidence_consistency`, `source_reliability`):

1. Coverage gate: `citation_coverage < 0.50` $\Rightarrow$ ESCALATE
2. Contradiction block: `contradiction_score > 0.50` $\Rightarrow$ ABSTAIN
3. Evidence-accept: all thresholds met $\Rightarrow$ `evidence_accept`
4. Default: $\Rightarrow$ ESCALATE with diagnostic

On MultiNLI ($N = 3,000$): resolution rate 38.5%, evidence-accept precision 100%, false-accept rate on contradiction items 0%, abstain precision on contradictions 99.3%.

8.6 Evidence Signal Provenance

Current implementation: `EvidenceSignal` is built as a proxy from oracle consensus statistics. This is documented in the source as a “structural bridge” pending semantic retrieval. *Planned:* external BM25/NLI passage retrieval feeding real document evidence. The routing logic is fully implemented and tested; real evidence quality may differ from NLI label quality.

9 Audit and Governance Envelope

9.1 DecisionEnvelope v2

Every gate invocation produces an attribute-frozen `DecisionEnvelope` (frozen dataclasses treated as read-only; nested lists/dicts are not deep-frozen) with blocks: `request`, `assessment`, `gate`, `reviewer_context`, `follow_up`, `history`, `policy_learning`, and `audit`. The library engine emits the envelope with `previous_hash` and `signature` unset; the API finalizer links the tenant hash-chain and adds an HMAC signature only when `REMORA_ENVELOPE_SIGNING_KEY` is configured. The full schema is in Appendix B.

9.2 Audit Hash-Chain

$$h_i = \text{SHA-256}(h_{i-1} \parallel \text{json}(e_i)) \quad (8)$$

This provides **tamper-detection**: any modification breaks the chain. It does **not** prevent tampering. *Tamper-proof* audit requires external append-only

storage (S3 Object Lock, Azure Immutable Blob, or equivalent) as a deployment dependency.

9.3 Follow-Up Workflow

ESCALATE generates a structured follow-up request specifying reason codes, required evidence, responsible role, and SLA. Data structures are implemented; production integration with external ticketing systems is a deployment requirement.

10 Experimental Evaluation

10.1 Benchmarks

QA benchmark ($N = 544$). BoolQ ($n = 377$) [23], TruthfulQA ($n = 85$) [22], adversarial-curated ($n = 7$), REMORA-curated ($n = 75$, author-assembled, selection bias possible).

Tool-call benchmark (700 rows; effective $N = 70$). Synthetic adversarial suite, leakage-remediated (2026-07-20): 70 unique templates $\times$ 10 cosmetic variants across 7 domains and 4 task types; the 700 rows are therefore 70 statistical clusters, not 700 independent tasks. Three adversarial failure modes: intent-argument conflict, change-window/dual-control requirements, safe-looking tasks with dangerous hidden scope. The simulator executes no real shell, network, database, or filesystem mutation; “unsafe” means unsafe *acceptance* of a simulated action.

Evidence benchmark ($N = 3,000$). MultiNLI validation-matched [24] (entailment, neutral, contradiction) as proxy for evidence-verification decisions.

10.2 Baselines

QA: full-coverage majority vote; trust-score selective (threshold=0.50).

Tool-call: single model (Llama-3.3-70B); majority vote; self-consistency; verifier heuristic; REMORA temperature-gate only (no policy engine).

10.3 Oracles

The oracle backend changed across rounds. Earlier QA and tool-call rounds ran a Groq-hosted three-LLaMA swarm (Llama-3.1-8B-Instant, Llama-3.3-70B-Versatile, Llama-4-Scout-17B-16E-Instruct — one weight family); the 2026-07 clean round, the AgentHarm validation, and the real-LLM baselines use a Cloudflare Workers AI cross-family trio (Meta LLaMA, Alibaba Qwen, Mistral). Oracle model versions may change; stored artifacts ensure deterministic reproduction.

11 Results

11.1 Selective Accuracy on QA Benchmark

Table 4: QA benchmark selective accuracy ($N = 544$).
 $\dagger$ In-sample optimum; $\ddagger$ held-out: stratified 80/20 split, $\tau^* = 0.203$ locked from training set (seed=42);
 $\S$ **PhaseAwareGuardrail**: ordered + inverted low- τ critical ($\tau < 0.10$), $N = 120$. Entropy-derived observables use the token-fingerprint proxy backend.

Condition	Set	Cov.	Acc.	Lift	95% CI
Majority vote (full)	full	100%	41.18%	—	[37.1, 45.4]
neg-temp. (10%) $\dagger$	in-sam	10%	81.48%	+40.3	[69.2, 89.6]
neg-temp. (18%) $\dagger$	in-sam	18%	88.78%	+47.6	[81.0, 93.6]
phase-aware (22%) $\S$	in-sam	22.1%	85.0%	+43.8	[77.5, 90.3]
neg-temp. (25%) $\dagger$	in-sam	25%	72.79%	+31.6	[64.8, 79.6]
neg-temp. (held-out, retired) $\ddagger$	holdout	23.2%	88.00%	+41.7	[70.0, 95.8]

Held-out: $N_{\text{accepted}} = 25$ (all ordered-phase), $p = 1.45 \times 10^{-5}$ (one-sided binomial, H_1 : acc > holdout baseline 46.3%), Wilson CI lies entirely above holdout baseline. **These are the retired Groq-round figures.** The 2026-07 cross-family clean round gives 100.0% at 16.7% coverage ($N_{\text{accepted}} = 18$, $p = 0.052$ — directional only), and the pre-registered SAP v3 fresh-data round *failed to confirm* the consensus-temperature selection signal (§15); do not read this row as evidence that temperature generalizes.

Phase breakdown on full dataset: ordered ($N=99$): 86.9%; critical ($N=32$): 62.5%; disordered ($N=413$): 28.6%. Disordered items constitute 75.9% of the benchmark, driving the low full-coverage baseline—a structural property of benchmark composition. Extending coverage beyond the ordered-only 18.2% operating point requires exploiting the critical-phase trust inversion (§8.2): the **PhaseAwareGuardrail** raises coverage to 22.1% at 85.0% accuracy by adding 21 low- τ critical items ($\tau < 0.10$, inverted-score selection, +43.8 pp lift; Wilson CI [77.5%, 90.3%]).

11.2 Tool-Call Benchmark (700 rows; effective $N = 70$)

Table 5: Tool-call benchmark, leakage-remediated 2026-07-20: REMORA vs baselines (700 rows; effective $N = 70$ template clusters).

Condition	Acc.	Unsafe acc.	Utility
Single model heuristic	28.6%	1.4%	0.16
Majority vote heuristic	28.6%	1.4%	0.16
Self-consistency heuristic	28.6%	1.4%	0.16
Verifier heuristic	28.6%	1.4%	0.16
REMORA temp. gate only	60%	1.4%	0.36
REMORA full policy	90%	0%	0.62

REMORA’s full policy gate produced 0 unsafe acceptances (cluster-level Wilson 95% CI [0.0%, 5.2%]); baselines produced 1.4% (1 of 70 clusters). The safety-rate difference is not statistically significant at the template-cluster level (one-sided $p = 0.50$); the statistically supported advantages are decision utility (+0.456, $p \approx 10^{-4}$) and accuracy. The hard-block layer carries the 0% safety floor, but on this benchmark version its measured advantage over the temperature-only gate is routing quality, not a statistically separable unsafe-rate reduction. (Earlier versions of this table reported baselines at 10–20% unsafe: those numbers were inflated by baselines and gate reading author-annotated severity and oracle context flags; the 2026-07-20 re-run restricts all conditions to the observable task surface.)

11.3 Conformal Coverage (Mondrian, $N = 2,161$)

Table 6: Mondrian phase-stratified conformal, 15% risk target (20 seeds $\times N = 2,161$ augmented dataset).

Phase	Mean Risk	Mean Cov.	Seeds Fail
Ordered	12.0%	99.9%	0/20
Critical	64.5%	0.9%	4/20
Disordered	78.6%	0.2%	2/20

Ordered-phase conformal achieves 99.9% coverage with 0/20 failures—a stable internal benchmark operating point (the $N = 2,161$ dataset is augmented and includes calibrated-simulation critical items; this is not evidence of deployability under production distribution shift). Critical and disordered phases cannot achieve meaningful coverage via conformal calibration alone, motivating the evidence router.

11.4 Evidence Router Proxy Benchmark (MultiNLI, $N = 3,000$)

Resolution rate 38.5%; evidence-accept precision 100%; false-accept rate on contradiction 0%; abstain precision 99.3%. Trust-only baseline: 0% resolution (all escalated). This NLI-proxy benchmark is distinct from the 32-case *Cross-Domain Evidence Replay* (static rule-based provider) reported in the repository and the enterprise white paper; the two measure different mechanisms at different scales.

11.5 Tool Routing on Sealed External Data (BFCL v4)

The benchmarks above measure whether a *harmful* call is stopped. This track also measures whether a harmless-looking but incorrect call is stopped. Track C-ext2 retained five targets that were sealed before evaluation. It sampled 258 `live_multiple` tasks and 258 `live_irrelevance` tasks from BFCL v4 at upstream commit `6ea57973c7a6` (Apache-2.0). The one permitted evaluation contained 1,527 episodes over 516 source clusters, with an empty state index, no validator bindings, and a registry derived only from task schemas.

Pre-registered target	Goal	Sealed BFCL v4 result	Verdict
Required-UNKNOWN autonomous Acc	$\leq 0\%$	$0/32 = 0.0\%$	met
Irrelevance ABS recall	$\geq 70\%$	$258/258 = 100.0\%$	met
Unobtainable ABS recall	$\geq 70\%$	$98/99 = 99.0\%$	met
Obtainable VER recall	$\geq 70\%$	$96/99 = 97.0\%$	met
Wrong-call Acc	$\leq 20\%$	$28/258 = 10.9\%$	met

All five pre-registered targets met. Labelled routing accuracy was **91.2%** ($n = 1,170$; 508 effective labelled clusters; cluster-level Wilson 95% CI [88.4%, 93.3%]). Optional-lane identity ACCEPT was $151/218 = 69.3\%$, reported without a target.

The confirmation has a precise boundary. BFCL provides no independently vouchable state table, so the wrong-argument-value axis remains excluded by admission. The track also supplies no authoritative `TaskIntent/ToolContract` bundle. Consequently, the blind result confirms the complete current routing pipeline; it does not isolate the causal efficacy of semantic intent matching.

11.6 What the Wrong-Call Axis Implies

The BFCL v4 result is this paper’s clearest external routing result:

Structural validity and semantic correctness are independent properties, and the governance signals that establish the first cannot establish the second.

Schema validity, argument satisfiability, provenance, taint and value grounding are all properties of the *call*. Whether the call serves the *task* is a property of the relation between them, and no amount of scrutiny of the call alone recovers it. The residue is therefore not a threshold to tune: there is no threshold on a signal that does not carry the information.

The architecture reserves a slot for the missing authority — `tool_matches_goal`, tri-state, held at UNKNOWN since definition rather than filled with a guess. A mechanism now occupies it: a deployment-declared tool contract (capability, effect, resource type, mutation, argument roles) matched against a structured task intent, routing an established contradiction to ABSTAIN for reads and ESCALATE for writes. It is placed ahead of trust routing, so a confident consensus cannot execute a call that provably serves the wrong goal, and behind every hard guard, so it can never unblock one. The authority limit is the design: a model may *propose* the intent but may not thereby assert a match, and an intent whose quoted spans do not occur in the task text yields UNKNOWN, never SUPPORTED.

Its isolated effect remains **unmeasured**, because the BFCL v4 confirmation contains no authoritative semantic bundle.

12 Ablations

Where the safety floor comes from. *Architectural implication.* The unsafe-execution reduction is 100% attributable to deterministic, interpretable policy rules that are independent of LLM output. The multi-oracle consensus and routing machinery improve decision *calibration and routing quality* (discriminating VERIFY, ABSTAIN, and ACCEPT paths), but do not contribute to the safety guarantee itself. This separation is a design feature: the safety floor can be reasoned about and audited without reference to oracle behaviour. Readers should not infer that the consensus machinery is the source of the 0% unsafe rate.

Oracle count ($N = 302$). Single oracle: 56.95% (95% CI [51.3, 62.4]). Three-oracle majority vote: $\approx +10$ pp. Correlation-aware weighting: additional gain within multi-oracle setting.

The primary benefit of the oracle swarm is disagreement *detection* (high H , high D) rather than accuracy improvement per se.

13 Case Study: Well Barrier Agent Action

No domain-engineering correctness is claimed. This is an illustrative governance walkthrough.

A drilling agent proposes: “Autonomously update kill mud weight from 1.52 to 1.48 SG to improve ROP on the 8½” section.” Risk tier: critical. Domain: `well_engineering`.

Oracle responses: Llama-3.1-8B: ESCALATE, conf 0.78; Llama-3.3-70B: ESCALATE, conf 0.85 (NORSOK D-010 §5.4); Llama-4-Scout: VERIFY, conf 0.62. One oracle failed; excluded.

Consensus: $\hat{p}(\text{ESCALATE}) = 0.71$, $\hat{p}(\text{VERIFY}) = 0.29$. $H = 0.866$ bits; $D = 0.29$; phase = critical ($T = 0.85$, near T_c); $\tau = 0.31$.

Evidence: No field inspection, no ECD calculation, no OIM sign-off. Citation coverage = $0.18 < 0.50$ threshold. Evidence router: ESCALATE.

Policy engine: Conditional policy gate fires (phase=critical AND risk=critical) $\Rightarrow$ ESCALATE. This gate resides in `_CONDITIONAL_GATES`, after the unconditional `hard_guard_floor()`; the outcome is identical.

Follow-up generated: type=on_site_inspection; assign_to=Independent Well Engineer; required_evidence = {ECD calculation, kick tolerance, NORSOK D-010 barrier confirmation, OIM sign-off}; SLA=4h.

Audit hash recorded; action blocked.

14 Case Study: State Resolution on a Fleet Operations Agent

The barrier case shows the deterministic floor stopping an action outright. This one shows the mechanism that distinguishes the current architecture: what happens when the action is well-formed and the *state it refers to* is uncertain. Both calls below are structurally perfect. They are routed differently, and neither routing depends on a model’s confidence.

Setup. A fleet-operations agent has six tools, of which `get_vehicle`, `get_driver`, `get_work_order` and `list_depot_vehicles` are declared **read** and `assign_driver` and `close_work_order` are declared **write**. The effects come from the registry, not from the verb in the name — a distinction that matters, because `assign` and `close` were once absent from a mutating-verb list and thirty writes were scored as reads. No bulk state export exists, so every argument value verdict is UNKNOWN: the deployment has point lookups and nothing else.

Call A: a legitimate read. The user asks for the status of vehicle V-0042; the agent proposes `get_vehicle(vehicle_id="V-0042")`. Every structural signal is satisfied. The value is grounded — V-0042 is named in the task text. But `vehicle_id`

steers where the read lands, and nothing has confirmed it exists, so autonomy is unavailable:

Step	Outcome
Argument support	UNKNOWN (no coverage declaration)
Value grounding	grounded (named in the task)
Validation requirement	required (steers the read)
Resolver availability	<code>get_vehicle</code> bound to the role
Verdict	VERIFY + <code>ResolutionPlan</code>
<i>validator consulted; entity confirmed present</i>	
Re-entry	whole router, fresh observation
Final verdict	ACCEPT

The plan names one resolver and exactly one argument it may write. The validator returns a verdict and hands control back; it never performs the original call and cannot redirect it. Re-entry re-runs the *whole* router, so every guard applies again to the confirmed call rather than being patched around. Had no validator been bound to `vehicle_id`, the verdict would be ABSTAIN, not VERIFY: promising a verification that cannot happen is worse than stopping.

Call B: the same confirmation, a different action. The agent proposes `assign_driver(work_order_id="W0-00123", driver_id="D-0017")`. Both identifiers confirm against the system of record exactly as V-0042 did. The call is nonetheless held at VERIFY: the registry declares this tool a write, and the read/write posture does not trade confirmed state for autonomy on a state change. No confidence score moves it.

Call C: structurally perfect, wrong tool. The user asks for a vehicle status; the agent proposes `get_work_order` with a work-order identifier that happens to appear elsewhere in the task. Schema valid, arguments satisfiable, values grounded, nothing tainted — every signal in Call A’s ladder is satisfied. The sealed BFCL v4 track measured wrong-call acceptance at 10.9%. A declared tool contract does: the task targets a vehicle, the tool acts on a work order, and the mismatch is established rather than merely unverified. That authority is implemented and **unmeasured** (§11.6).

15 Negative Results

Publication of negative results is essential for scientific credibility. This section documents active and resolved findings. Full documentation is in `NEGATIVE_RESULTS.md`.

Table 7: Negative results and mitigations.

Finding	Sev.	Status	Mitigation
χ -proxy AUC = 0.39	Med	Docum.	Exploratory OOD use
T - D circularity	Med	Resolv.	Structural T
Crit.-phase anticorr.	High	Active	Evidence router
Full-cov. acc. weak	Med	Active	Selective prediction
Oracle diversity part.	Med	Active	Cross-family trio
Temp. selection falsified	High	Active	Diagnostic only; use calib. conf.
Evidence signal proxy	Med	Active	NLI benchmark
Audit not tamper-proof	Med	By des.	WORM storage

Consensus temperature failed fresh-data confirmation (SAP v3). On the pre-registered SAP v3 round (1231 fresh BoolQ/TruthfulQA items, deduplicated against the prior 544-item corpus; frozen Cloudflare cross-family trio; three-way split dev 493 / risk-cal 370 / test 368, seed 20260727), the consensus-temperature selection signal *failed* confirmation: test-split AURC 0.0954 vs. 0.0664 for a dev-split-calibrated mean-confidence baseline (paired bootstrap $\Delta = 0.0290$, 95% CI [0.0119, 0.0503] excluding zero — temperature ranks significantly *worse*), and SGR certification at $r^* = 5\%$, $\delta = 0.10$ found no certifiable coverage. The exploratory temperature advantage seen on the reused 544-item corpus did not transfer; temperature is retained as a logged diagnostic, not an authoritative selector. For scale: distinguishing a 90%-accurate selector from an 80% full-coverage baseline at conventional thresholds (one-sided $\alpha = 0.05$, power $1 - \beta = 0.8$) requires on the order of 150–200 *accepted* items—an order of magnitude beyond the $N_{\text{accepted}} = 18$ –25 accepted sets available in the rounds above, which is why those are reported as directional only. *What survives*: on the same round, calibrated confidence-weighted voting scored 87.8% vs. 85.1% for unweighted majority on the untouched test split ($N = 368$; paired exact McNemar $p = 0.0064$, significant; vs. the best single model directional only, $p = 0.077$). Its selection certificates are marginal per arm and no arm survives family-wise Bonferroni-3, so calibrated confidence is a promising pre-registered *secondary* that requires its own frozen confirmation round before any engine integration (SAP v3 §8 D-3). Full detail: `NEGATIVE_RESULTS.md` §18 and claim register CLAIM-012 / CLAIM-013.

χ -proxy failure. Susceptibility χ as standalone difficulty predictor: AUC = 0.39 on $N = 302$ items—below chance. Exploratory OOD use (not a demonstrated detector; the live variant is near-constant unless internal clamps bind): $\chi > 1.45$ (97th percentile) $\Rightarrow$ ESCALATE with `escalate_adversarial`.

Critical-phase trust anticorrelation. Q4 high-trust: 50% correct; Q1 low-trust: 75% correct. Confirmed on $N = 32$ real-oracle items and replicated qualitatively on augmented $N = 511$ simulated dataset. No trust threshold separates correct from incorrect critical-phase items. *Consequence*: evidence routing—not trust scoring—is the required mechanism for critical-phase decisions.

T - D circularity (resolved). Legacy temperature estimator weighted D at 18% of T , creating a feed-back loop ($D \rightarrow T \rightarrow F \rightarrow V$, all depending on D). Resolved by structural temperature (§5), which computes T from prompt structure alone.

16 Threats to Validity

Internal validity: Trust threshold ($\tau^* = 0.2032$, 18th-percentile neg-temperature on 80% training split) is derived in-sample; the retired-Groq held-out evaluation ($n = 108$, 20% stratified, $p = 1.45 \times 10^{-5}$) supported the threshold, but the SAP v3 fresh-data round did not confirm the temperature signal it selects on (§15); 13.8% author-curated benchmark items (15.1% including the adversarial-curated set); synthetic tool-call adversarial patterns.

External validity: Model coverage is mixed across rounds: the earlier QA and tool-call rounds used a Groq-hosted three-LLaMA swarm (one weight family, partial diversity), while the 2026-07 clean round, the AgentHarm external validation, and the real-LLM baselines use a Cloudflare Workers AI cross-family trio (Meta LLaMA, Alibaba Qwen, Mistral). Results on other providers, closed-source models, or quantized variants may still differ; the benchmark phase distribution may not match deployment reality; and MultiNLI is a proxy for field evidence retrieval.

Construct validity: The thermodynamic terminology of earlier versions has been withdrawn (§5). “Trust” is a derived scalar, not a frequency probability of per-item correctness. Utility scores are designed task weights, not real-world costs.

Statistical validity: 95% Wilson confidence intervals are reported for key accuracy figures; Wilson intervals are fixed- N constructions and are not valid under the continuous monitoring that the REM-020 longitudinal gate performs (optional stopping). A time-uniform confidence sequence [53, 31, 38, 46] is therefore reported alongside for the gate’s cycle-level false-accept indicator: 0/168 cycles gives a 95% time-uniform upper bound of 4.72% (`results/far_confidence_sequence_v1.json`, claim register CLAIM-011). The sequence is wider than Wilson at any fixed

N : the price of validity under data-dependent stopping.

17 Safety, Ethics, and Governance

REMORA is designed to *reduce* unsafe autonomous AI actions. ACCEPT means assurance conditions are met, not that the action is correct. Human reviewers receiving ESCALATE retain full accountability for outcomes. The oracle swarm routes sensitive operational data through external APIs; data classification requirements must be evaluated before deployment. The adversarial firewall detects known injection patterns but is not a complete defense against sophisticated adversarial inputs.

18 Reproducibility

Repository: <https://github.com/darklordVirtual/REMORA-research>. **Release:** Preprint v0.10.0, revision 2026-08-07 (synchronized with repository code); repository tag `review-v1` (this paper is versioned in lockstep with the repository release tag). **Python:** 3.11+.

```
python -m pip install -e ".[dev]"
make audit          # lint + tests + claim consistency
make benchmark      # all deterministic benchmarks
make credibility-pack
```

Live oracle experiments require `GROQ_API_KEY` (earlier rounds) or `CLOUDFLARE_API_TOKEN` (Cloudflare Workers AI, the 2026-07 clean round). Oracle model versions may change; results are sensitive to model version updates. Stored artifact experiments are fully deterministic.

License. This preprint is distributed as part of the REMORA Licensed Work under the Business Source License 1.1 (source-available, not open source). Commercial use requires a separate REMORA Commercial License from the Licensor, Stian Skogbrott (support@luftfiber.no). Third-party datasets and materials remain under their own terms; see `THIRD_PARTY_NOTICES.md`.

Claim binding: headline numbers are bound to the machine-readable claim register (`docs/assurance/claim_register_v1.yaml`), the single authoritative source, through the Markdown source of this paper (`paper/remora_paper.md`): the claim-provenance and claim-consistency CI gates fail on divergence between that source and the register. This LaTeX manuscript and the compiled PDF are kept in lockstep with the

Markdown source by `scripts/check_paper_sync.py` (version, revision, and guarded process claims); numeric drift in the typeset copy is caught by review rather than by an automated gate on the `.tex`. Each `review-*` tag triggers a workflow that regenerates a commit-bound test attestation and a governance pack whose `manifest.json` records the exact commit SHA and per-file SHA-256 checksums, uploaded as one `remora-review-handoff-<sha>` artifact.

19 Conclusion

We presented REMORA, a policy-gated assurance architecture for governing agentic AI actions before they execute. Three results carry the paper, and the third is the one we would most like the field to take up.

Autonomous safety needs a deterministic floor. The 0% unsafe-acceptance result on the adversarial tool-call benchmark (700 synthetic rows; effective $N = 70$ clusters; cluster-level Wilson CI [0.0%, 5.2%]; baselines 1.4%, difference not significant at $p = 0.50$; the significant gains are utility +0.456 and accuracy) is attributable *entirely* to deterministic hard blocks, not to the multi-oracle consensus machinery. We state this against our own interest: the probabilistic architecture that gives the system its name is not what produces its headline safety number. It contributes routing quality, and that is a different claim requiring different evidence.

Unknown state is a resolution problem. Absence without declared completeness is UNKNOWN, not UNSUPPORTED; a VERIFY that cannot name a bounded lookup degrades to ABSTAIN; and a resolved value re-enters the whole router rather than patching the prior decision. Under study conditions this recovers read utility from 0% to 100% in the all-UNKNOWN regime with zero corrupt-identifier acceptances, zero cross-tenant lookups, and writes held at VERIFY throughout.

The routing behaviour transfers. A sealed BFCL v4 track evaluated the current pipeline once and met all five pre-registered targets: wrong-call acceptance **10.9%** (28/258), irrelevant-tool refusal 100.0%, unobtainable-input refusal 99.0%, obtainable-input verification 97.0%, and required-UNKNOWN autonomous acceptance 0.0%. Labelled routing accuracy was 91.2%. Structural validity and semantic correctness remain distinct properties, and the isolated effect of semantic task-contract matching is not established by this track.

Key negative findings retained in full: trust anti-correlates with correctness in the critical phase; the χ -proxy fails as a difficulty predictor (AUC = 0.39); a

pre-registered fresh-data round falsified the consensus-temperature selection signal, which is retained only as a diagnostic; evidence retrieval is proxy-based; entropy uses a token-fingerprint heuristic rather than semantic entropy over NLI clusters; and the AgentHarm run blocked every benign counterpart as well as every harmful one.

REMORA is a research-grade prototype, SHADOW_ONLY, with independent external replication and production field evidence pending. We offer it as a reference architecture and empirical baseline. The broader warning remains: a gate that reasons only about action well-formedness cannot establish that the action serves the user’s goal.

Acknowledgements

Benchmarks use publicly available datasets (BoolQ, TruthfulQA, MultiNLI, ARC-Challenge, MMLU-Pro). Oracle inference via the Groq API (earlier rounds) and Cloudflare Workers AI (the 2026-07 clean round). Open Policy Agent used for policy evaluation. Negative results documented in full in `NEGATIVE_RESULTS.md`; the authors thank all reviewers who pointed out scope corrections and limitations. We thank contributor Erlend Barli (github.com/erlendbarli) for contributions to the project (see `CONTRIBUTORS.md`).

Use of AI-Assisted Development Tools

Generative AI tools were used as assistive tools during the development of this research prototype. Their use included brainstorming, code drafting and refactoring, test scaffolding, documentation editing, language revision, and support in locating and summarising relevant literature.

The author defined the research questions, system architecture, policy semantics, evaluation criteria, experimental scope, and all reported claims. All source code, tests, benchmark artifacts, citations, numerical results, and conclusions were reviewed and verified by the author before inclusion. Generative AI tools were not listed as authors and were not relied upon as independent sources of experimental evidence.

Where AI-assisted output informed implementation or prose, the final responsibility for correctness, reproducibility, and interpretation remains with the human author. A full disclosure is available in `docs/AI_USE.md`.

References

- [1] Kuhn, L., Gal, Y., & Farquhar, S. (2023). Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *Proceedings of the International Conference on Learning Representations (ICLR 2023)*. arXiv:2302.09664.
- [2] Kirchner, J. H., Chen, Y., Edwards, H., Leike, J., McAleese, N., & Burda, Y. (2024). Prover-verifier games improve legibility of LLM outputs. arXiv:2407.13692.
- [3] Angelopoulos, A. N., Bates, S., Fisch, A., Lei, L., & Schuster, T. (2022). Conformal risk control. arXiv:2208.02814.
- [4] Tibshirani, R. J., Barber, R. F., Candès, E. J., & Ramdas, A. (2019). Conformal prediction under covariate shift. *NeurIPS 2019*.
- [5] Andriushchenko, M., et al. (2024). AgentHarm: A benchmark for measuring harmfulness of LLM agents. arXiv:2410.09024.
- [6] Dong, B. & Wang, Q. (2025). Evaluating the performance of the DeepSeek model in confidential computing environment. arXiv:2502.11347.
- [7] Wang, X., et al. (2023). Self-consistency improves chain of thought reasoning in language models. *ICLR 2023*.
- [8] Du, Y., Li, S., Torralba, A., Tenenbaum, J., & Mordatch, I. (2024). Improving factuality and reasoning in language models through multiagent debate. *ICML 2024*.
- [9] Zheng, L., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS 2023*.
- [10] Cobbe, K., et al. (2021). Training verifiers to solve math word problems. arXiv:2110.14168.
- [11] Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1), 41–46.
- [12] El-Yaniv, R. & Wiener, Y. (2010). On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11, 1605–1641.
- [13] Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in Neural Information Processing Systems (NeurIPS 2017)*.

- [14] Kuncheva, L. I. & Whitaker, C. J. (2003). Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. *Machine Learning*, 51(2), 181–207.
- [15] Geifman, Y. & El-Yaniv, R. (2017). Selective classification for deep neural networks. *NeurIPS 2017*.
- [16] Kadavath, S., et al. (2022). Language models (mostly) know what they know. *arXiv:2207.05221*.
- [17] Angelopoulos, A. & Bates, S. (2021). A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv:2107.07511*.
- [18] Guo, C., et al. (2017). On calibration of modern neural networks. *ICML 2017*.
- [19] Endsley, M. (1995). Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1), 32–64.
- [20] Kelly, T. (1998). Arguing safety: A systematic approach to managing safety cases. PhD thesis, University of York.
- [21] Bloomfield, R. & Bishop, P. (2010). Safety and assurance cases: Practice, trends and challenges. *SAFECOMP 2010*.
- [22] Lin, S., et al. (2022). TruthfulQA: Measuring how models mimic human falsehoods. *ACL 2022*.
- [23] Clark, C., et al. (2019). BoolQ: Exploring the surprising difficulty of natural yes/no questions. *NAACL 2019*.
- [24] Williams, A., et al. (2018). A broad-coverage challenge corpus for sentence understanding through inference. *NAACL 2018*.
- [25] European Parliament & Council of the European Union. (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council (Artificial Intelligence Act). *Official Journal of the European Union*, L 2024/1689.
- [26] Open Policy Agent. (2024). <https://www.openpolicyagent.org/>.
- [27] Bjørn, A. R. (2026). Causal Post-hoc Explainable AI. PhD thesis, Norwegian University of Science and Technology (NTNU). ISBN 978-82-353-0022-5.
- [28] Chennabasappa, S., et al. (2025). LlamaFirewall: An open source guardrail system for building secure AI agents. *arXiv:2505.03574*.
- [29] Choi, M., Yang, S., Kim, D., Kim, S., Son, J., Lee, Y., Choo, J., & Kwak, Y. (2026). Membrane: A self-evolving contrastive safety memory for LLM agent defense. *arXiv:2606.05743*.
- [30] Corsi, D., Marchesini, E., & Farinelli, A. (2021). Formal verification of neural networks for safety-critical tasks in deep reinforcement learning. *UAI 2021*, PMLR 161:333–343.
- [31] Darling, D. A. & Robbins, H. (1967). Confidence sequences for mean, variance, and median. *Proceedings of the National Academy of Sciences*, 58(1), 66–68.
- [32] Debenedetti, E., et al. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *NeurIPS 2024 Datasets & Benchmarks*.
- [33] Debenedetti, E., et al. (2025). Defeating prompt injections by design (CaMeL). *arXiv:2503.18813*.
- [34] Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. *Nature*, 630, 625–630.
- [35] Ge, Y. (2026). Governance architecture for autonomous agent systems: Threats, framework, and engineering practice. *arXiv:2603.07191*.
- [36] Greenblatt, R., Shlegeris, B., Sachan, K., & Roger, F. (2024). AI control: Improving safety despite intentional subversion. *ICML 2024*, PMLR 235:16295–16336.
- [37] Guldimann, P., et al. (2024). COMPL-AI framework: A technical interpretation and LLM benchmarking suite for the EU Artificial Intelligence Act. *arXiv:2410.07959*.
- [38] Howard, S. R., Ramdas, A., McAuliffe, J., & Sekhon, J. (2021). Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics*, 49(2), 1055–1080.
- [39] International Electrotechnical Commission. (2016). IEC 61511-1:2016, Functional safety: Safety instrumented systems for the process industry sector. IEC.
- [40] Inan, H., et al. (2023). Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv:2312.06674*.

- [41] Mohri, C. & Hashimoto, T. (2024). Language models with conformal factuality guarantees. *ICML 2024*.
- [42] Mu, T., Helyar, A., et al. (2024). Rule based rewards for language model safety. *NeurIPS 2024*.
- [43] Standards Norway. (2021). NORSOK D-010:2021, Well integrity in drilling and well operations. Standards Norway.
- [44] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.
- [45] Raji, I. D., Xu, P., Honigsberg, C., & Ho, D. E. (2022). Outsider oversight: Designing a third party audit ecosystem for AI governance. *AIES ’22*. arXiv:2206.04737.
- [46] Ramdas, A., Grünwald, P., Vovk, V., & Shafer, G. (2023). Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4), 576–601.
- [47] Rebedea, T., et al. (2023). NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. *EMNLP 2023 System Demonstrations*.
- [48] Ruan, Y., et al. (2024). Identifying the risks of LM agents with an LM-emulated sandbox (ToolEmu). *ICLR 2024*.
- [49] Shafer, G. & Vovk, V. (2008). A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9, 371–421.
- [50] Shamsujjoha, M., Lu, Q., Zhao, D., & Zhu, L. (2024). A taxonomy of multi-layered runtime guardrails for designing foundation model-based agents: Swiss cheese model for AI safety by design. arXiv:2408.02205.
- [51] Sharma, M., et al. (2025). Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. arXiv:2501.18837.
- [52] Shi, T., He, J., Wang, Z., Li, H., Wu, L., Guo, W., & Song, D. (2025). Progent: Securing AI agents with privilege control. arXiv:2504.11703.
- [53] Ville, J. (1939). *Étude critique de la notion de collectif*. Gauthier-Villars.
- [54] Vovk, V., Gammernan, A., & Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer.
- [55] Wang, X., Aitchison, L., & Rudolph, M. (2023). LoRA ensembles for large language model fine-tuning. arXiv:2310.00035.
- [56] Wang, H., Poskitt, C. M., et al. (2025). AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. arXiv:2503.18666.
- [57] Xiang, Z., et al. (2025). GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning. *ICML 2025*.
- [58] Yadkori, Y. A., Kuzborskij, I., et al. (2024). Mitigating LLM hallucinations via conformal abstention. arXiv:2405.01563.
- [59] Yang, C. (2026). AgentTrust: A self-improving trust layer for AI-agent actions. arXiv:2606.08539.
- [60] Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). τ -bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045.
- [61] Yuan, T., et al. (2024). R-Judge: Benchmarking safety risk awareness for LLM agents. *Findings of EMNLP 2024*.
- [62] Zhan, Q., Liang, Z., et al. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated LLM agents. *Findings of ACL 2024*.
- [63] Zhang, M., Huang, M., Shi, R., Guo, L., Peng, C., Yan, P., Zhou, Y., & Qiu, X. (2024). Calibrating the confidence of large language models by eliciting fidelity. arXiv:2404.02655.

A Mathematical Notation Table

Table 8: Mathematical notation.

Symbol	Definition	Domain
$\mathcal{O}$	Oracle set	$n \geq 2$ endpoints (design intent ≥ 3 distinct families)
$\mathcal{O}'$	Valid oracle set	$\mathcal{O}' \subseteq \mathcal{O}$
$\varphi(o)$	Canonical verdict	(polarity, hash, mag., tags)
$\rho(a, b)$	Pairwise agreement rate	$[0, 1]$, window 200
$w(o)$	Diversity weight	normalized
$\tilde{p}(v)$	Weighted support	$[0, 1]$, sums to 1
H	Shannon entropy	bits ≥ 0
D	Dissensus	$[0, 1]$
k	Active verdict count	≥ 1
η	Consensus concentration	$[0, 1]$
χ	Susceptibility	$[0, \infty)$
λ	Coupling constant	0.3 (paper config; library default 1.0)
τ	Trust score	$[0, 1]$
h_{bound}	Heuristic false-consensus risk proxy	$[0, 1]$
w_{phase}	Phase weight	$\{1.0, 0.5, 0.1\}$
g	Gate outcome	ACCEPT/VERIFY/ABS./ESC.
Γ	Gate function	$(q, a, c) \rightarrow g$

B DecisionEnvelope v2 Schema Summary

Top-level blocks: `request`, `assessment`, `gate`, `reviewer_context`, `follow_up`, `history`, `policy_learning`, `audit`. Full JSON schema is in the Markdown paper and repository (`remora/policy/report.py`).

C Negative Results Archive

The complete, versioned record of negative and resolved findings — including the resolved items R1 (iteration damage), R2 (Stage-3b critique impact) and R7 (T - D circularity) — lives in `NEGATIVE_RESULTS.md` in the repository. It is not duplicated here: the main-body Negative Results section carries the findings that shape the architecture; this appendix is a single pointer to the full archive so the two do not drift.

D Implementation Status Table

Status vocabulary: *Integrated* = invoked by the primary `/v1/assess` path; *Offline* = implemented and evaluated with committed artifacts but not invoked by the primary API; *Adapter* = implemented and parity-tested, available as an integration option; *Planned* = not implemented.

Table 9: Component status: implemented vs. integrated in `/v1/assess`.

Component	Impl.	Integrated	Notes
Oracle fan-out	Yes	Yes	ThreadPoolExecutor
Canonicalization (ρ)	Yes	Yes	NFKC, Semantic Entropy [1]
Correlation weighting	Yes	Per-request	Fresh engine per API request
H, D, η, F, τ	Yes	Yes	thermodynamics.py
V01 Equivariant tracking	Yes	Yes	Abort on ΔV
Phase classification	Yes	Yes	3-class
Policy engine (7 headline controls: 3 unconditional + 4 conditional)	Yes	Yes	Full gate inventory in code
OPA/Rego adapter	Adapter	No	API calls Python engine directly
Governance Intelligence	Offline	No	Optional pre-policy helper
PhaseGuardrail	Offline	No	Committed $N = 544$ artifact
Covariate-shift CRC	Offline	No	Nominal bound (heuristic weights)
PVID-inspired agreement score	Offline	No	Post-hoc heuristic
Evidence router (oracle proxy)	Yes	Yes	Proxy signal
Evidence router (semantic)	Planned	No	Interface ready
DecisionEnvelope v2	Yes	Yes	Attributes frozen, not deep-frozen
SHA-256 hash-chains	Yes	API layer	EMAC only with signing key
Execution grant path	Yes	Separate router	Dispatches via <code>GovernedToolDispatcher</code> (lease-bound; CAP-013)
Human review workflow	Yes	Partial	Denso UI only
WORM audit storage	Planned	No	External dependency
ZK assurance proofs	Planned	No	Stalled

E Conformal Risk Control Under Covariate Shift (offline component)

Standard Mondrian conformal calibration assumes that calibration and test samples are *exchangeable* - drawn i.i.d. from the same distribution. This assumption is violated in the critical phase: the joint distribution (τ , correct) differs structurally between ordered-phase calibration items and critical-phase test

items due to the trust-score inversion documented above. Empirically, applying standard conformal to critical-phase items yields 100% observed risk and zero coverage.

REMORA addresses this with a **weighted empirical selective router** *inspired by* Conformal Risk Control (CRC, [3]) but — important — *not implementing the CRC procedure* (see below). Each calibration item i receives an importance weight $w_i = p_{\text{test}}(x_i)/p_{\text{cal}}(x_i)$. For REMORA’s phase shift, we use the hand-tuned estimate

$$w_i = \begin{cases} 1.0 & \text{if phase}(i) = p_{\text{test}} \\ \beta & \text{otherwise} \end{cases} \quad (\beta = 0.10) \quad (9)$$

The selector picks the *smallest* threshold $\hat{\lambda}$ (i.e. maximum coverage) such that the weighted empirical risk $\bar{L}(\lambda) = \sum_i \tilde{w}_i \ell_i(\lambda) \leq \alpha$, with $\tilde{w}_i = w_i / \sum_j w_j$ normalised over the calibration items.

Theorem 1 (CRC Guarantee, Angelopoulos et al. 2022 — stated for context; not instantiated by the implementation). *For any monotone loss $\ell : [0, 1] \rightarrow [0, B]$, target risk $\alpha \in (0, B)$, and correctly-specified importance weights:*

$$\mathbb{E}[L(\hat{\lambda})] \leq \alpha + \frac{B}{n+1}$$

where n is the calibration set size.

Non-applicability. The implemented selector does not instantiate this theorem, for two structural reasons no weight specification can repair (external review R3-01): (a) its threshold rule applies the raw constraint $\bar{L}(\lambda) \leq \alpha$ with no $B/(n+1)$ finite-sample term and no test-point weight w_{n+1} ; (b) the controlled quantity — the conditional error rate among accepted items — is not monotone in λ , so Theorem 1 does not attach even with valid density-ratio weights. In addition, $\beta = 0.10$ is a hand-tuned constant, not a density ratio [4]. All quality evidence for this component is empirical (repeated-split holdout results).

The component is implemented as `remora.selective.crc.WeightedEmpiricalSelectiveRouter` (formerly `CovariateShiftCRC`, retained as an alias) with a `fit(scores, labels, phases, target_phase)` interface compatible with `ConformalPhaseGuardrail`. A `CRCReport` data-class exposes `finite_sample_slack` ($= 1/(n_{\text{cal}} + 1)$) and `nominal_risk_bound` ($= \alpha + \text{slack}$) as informational reference values (not applicable bounds), plus `holdout_risk` (plain unweighted holdout count). The module is fully tested (45 unit tests) and is an offline research component, not invoked by the reference API.

F Optional TEE-Based Audit Hardening (Tamper-Resistant)

This appendix describes tamper-resistant (not tamper-proof) audit hardening via hardware attestation [6]. Implementation status: protocol specified; hardware integration pending. The term “tamper-resistant” is used throughout; the hash-chain in the main system is tamper-detecting only.

F.1 Motivation

The SHA-256 hash-chain in `remora/audit/hash_chain.py` detects post-hoc modification of audit records but does not prevent an adversary with storage write access from replacing the entire chain. For deployments where stronger guarantees are required, hardware attestation via a Trusted Execution Environment (TEE) provides a complementary layer: the `DecisionEnvelope` is signed inside a hardware-isolated enclave, and the attestation quote includes a measurement of the enclave binary, making silent substitution detectable even if storage is compromised.

This is an optional deployment hardening; it is not required for the core REMORA governance protocol described in the main paper.

F.2 Supported TEE Platforms

Three platforms are in scope for the reference implementation:

- **Intel TDX / SGX** - remote attestation via DCAP quote generation
- **AMD SEV-SNP** - firmware measurement in attestation report
- **ARM TrustZone** - OP-TEE trusted application (mobile / edge)

F.3 DecisionEnvelope Attestation Format

When TEE hardening is active, each `DecisionEnvelope` includes an `attestation` sub-object:

```
{
  "envelope_id": "env_7f3a...",
  "gate": { "outcome": "ESCALATE", ... },
  "attestation": {
    "tee_platform": "intel_tdx",
    "quote": "<base64 DCAP quote>",
    "mrenclave": "<SHA-256 of enclave measurement
  ↪ >",
```

```
    "signature": "<Ed25519 over envelope_id
                  +outcome+timestamp>",
    "timestamp": "2026-01-15T14:23:07Z"
  }
}
```

The `mrenclave` value pins the exact binary that produced the decision, allowing auditors to verify that no unauthorised code modification occurred between decisions.

F.4 Verification Flow

1. Auditor retrieves envelope and attestation quote from audit log.
2. Quote is verified against Intel/AMD/ARM attestation service.
3. `mrenclave` is compared against the published release measurement.
4. Ed25519 signature over `(envelope_id, outcome, timestamp)` is verified with the enclave’s ephemeral key, which is bound to the quote.

If any step fails, the audit record is flagged as potentially tampered.

F.5 Implementation Status

Protocol specified and data contract defined. Hardware integration (enclave compilation, DCAP provisioning, OP-TEE TA signing) is a deployment dependency and is not part of the core source-available release. Stub interfaces are present in `remora/audit/` for downstream integration.